Examen de Programación Orientada a Objetos

Proyecto: Sistema Modular de Póker Clásico (5 Cartas) — Grupo B

Objetivo del Proyecto

Desarrollar una aplicación de consola en C# que simule una mesa de Póker Clásico (Showdown de 5 Cartas) aplicando los principios fundamentales de la Programación Orientada a Objetos: Clases, Encapsulamiento, Constructores (con sobrecarga), Herencia, Polimorfismo, Abstracción e Interfaces.

Requisito de Diseño de Software

La dirección técnica del club de cartas exige que el sistema sea altamente modular y desacoplado. El MotorJuego bajo ninguna circunstancia debe contener código condicional específico de cómo se juega el Póker; el motor debe ser un orquestador genérico que delegue la lógica del juego a través de un contrato formal (IReglasJuego).

Especificaciones Técnicas y Arquitectura Obligatoria

Debes organizar tu solución con las siguientes clases y responsabilidades exactas:

1. Enumeradores (Enums)

- **Palo:** Corazones, Diamantes, Treboles, Picas.
- **ValorCarta:** Dos = 2, Tres = 3, ..., Diez = 10, Jota = 11, Reina = 12, Rey = 13, As = 14.

2. Modelos (Modelos)

Carta

- **Propiedades encapsuladas:** Palo Palo { get; private set; } y ValorCarta Valor { get; private set; }.
- **Constructores (Sobrecarga de constructores):**
    - Constructor principal: Carta(Palo palo, ValorCarta valor).
    - Constructor sobrecargado: Carta(Palo palo, int valorNumerico).
- **Polimorfismo:** Sobreescribir override string ToString() para mostrar el valor y símbolo (ej: 10 ♥).
- **Renderizado:** Generar la visualización en formato ASCII con sus colores y caracteres Unicode.

Mazo

- **Encapsulamiento:** Lista privada de cartas (_cartas).
- **Constructor:** Llena las 52 cartas estándar combinando todos los palos y valores.
- **Métodos:**
    - Barajar(): Implementa el algoritmo de barajado aleatorio (Fisher-Yates).
    - TomarCarta(): Extrae y devuelve la carta superior.
- **Sobrecarga de Métodos (Overloading):**
    - Repartir(int cantidad): Devuelve una lista con la cantidad de cartas solicitadas.
    - Repartir(): Devuelve 1 sola carta.

Jugador

- **Propiedades:** Nombre, Mano (lista de cartas), Puntuacion (int), DetalleJugada (string).
- **Sobrecarga de Constructores:**
    - Jugador(string nombre)
    - Jugador(string nombre, int id)
- **Sobrecarga de Métodos:**
    - RecibirCarta(Carta carta)
    - RecibirCarta(List<Carta> cartas)

3. Abstracción y Contrato de Reglas (Estrategias)

IReglasJuego (Interfaz)

Contrato que define lo que cualquier conjunto de reglas debe proporcionar al motor:

```
C#
public interface IReglasJuego
{
    string NombreJuego { get; }
    string Descripcion { get; }
    void Repartir(Mazo mazo, List<Jugador> jugadores);
    int EvaluarPuntuacion(List<Carta> mano, out string descripcionJugada);
    string DeterminarGanador(List<Jugador> jugadores);
}
```

ReglasJuegoBase (Clase Abstracta - Herencia y Protecciones)

Clase base que implementa IReglasJuego:

- **Propiedades:** NombreJuego y Descripcion con nivel de protección protected set.
- **Constructor protegido:** protected ReglasJuegoBase(string nombreJuego, string descripcion) accesible solo mediante base(...).
- Define como abstract los métodos Repartir, EvaluarPuntuacion y DeterminarGanador.

ReglasPoker (Herencia y Polimorfismo)

- Hereda de ReglasJuegoBase (class ReglasPoker : ReglasJuegoBase).
- En su constructor llama a base("Póker Clásico (5 Cartas)", "...").
- **Repartir:** Reparte exactamente 5 cartas a cada jugador participante.
- **EvaluarPuntuacion:**
    - Clasifica la mano del jugador según la jerarquía tradicional de Póker:
        1. Escalera Real
        2. Escalera de Color
        3. Póker (4 cartas iguales)
        4. Full House (Trío + Pareja)
        5. Color (5 cartas del mismo palo)
        6. Escalera (5 cartas consecutivas)
        7. Trío (3 cartas iguales)
        8. Doble Pareja
        9. Pareja
        10. Carta Alta
    - Asigna una puntuación numérica ponderada y una descripción clara (ej: "Full House de Ases y Reyes", "Pareja de Jotas").
- **DeterminarGanador:** Compara las puntuaciones jerárquicas y declara al ganador con la mejor jugada de la mesa.

4. Motor de Juego Desacoplado (Motor)

MotorJuego

- **Inyección de Dependencias por Constructor:**

```
C#
public MotorJuego(IReglasJuego reglas)
```

- El motor almacena la regla en un campo privado _reglas y no debe saber que existe una clase llamada ReglasPoker.
- **Sobrecarga de Métodos:**
    - AgregarJugador(string nombre)
    - AgregarJugador(Jugador jugador)
- **Método EjecutarRonda():**
    - 1. Baraja el mazo.
    - 2. Llama a _reglas.Repartir(_mazo, _jugadores).
    - 3. Evalúa la puntuación llamando a _reglas.EvaluarPuntuacion(...).
    - 4. Muestra las cartas y resultados en consola.
    - 5. Llama a _reglas.DeterminarGanador(_jugadores) e imprime el veredicto final.

5. Programa Principal (Program.cs)

Configura la consola en UTF-8, crea una instancia de la regla, se la inyecta al motor, agrega 3 jugadores y ejecuta la ronda:

```
C#
using System.Text;

Console.OutputEncoding = Encoding.UTF8;

IReglasJuego regla = new ReglasPoker();
MotorJuego motor = new MotorJuego(regla);

motor.AgregarJugador("Jugador 1");
motor.AgregarJugador(new Jugador("Jugador 2", 2));
motor.AgregarJugador("Jugador 3");

motor.EjecutarRonda();
```

Criterios de Éxito

1. El proyecto debe compilar limpiamente sin errores ni advertencias.
2. Cada clase debe respetar estrictamente sus modificadores de acceso (public, private, protected).
3. El motor NO debe contener lógica de Póker; toda la lógica matemática reside en ReglasPoker.
4. Los naipes deben imprimirse de forma clara con sus símbolos Unicode (♥, ♦, ♣, ♠).